



COMP 412
FALL 2012

Introduction to Parsing

Comp 412

1st Hour Exam available today, 9/18/12
Due back @ 5PM on 9/27/12
Two hours; closed notes; closed book
Covers Lectures 1 through 10 (titled
Building Scanners on the web site)
Chapters 1 & 2, plus 13.1 — 13.3 in EaC2e

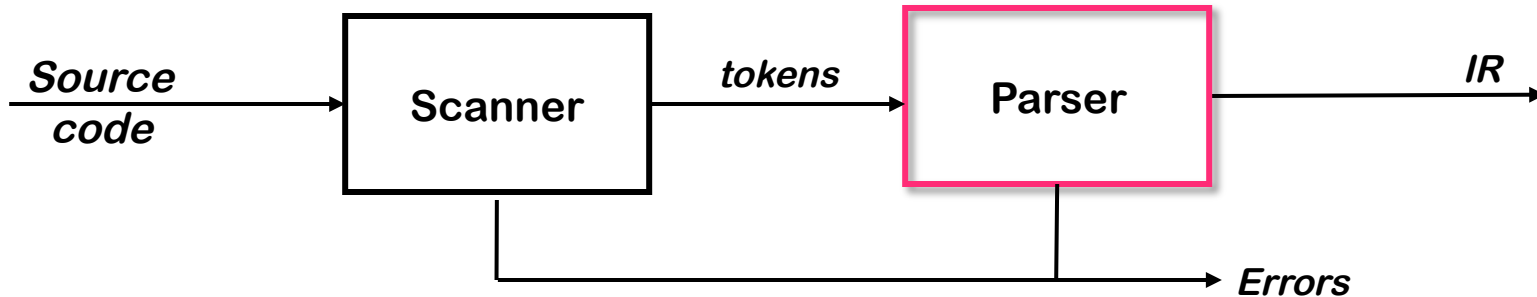
Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.



The Front End



Parser

- Checks the stream of words and their parts of speech (produced by the scanner) for grammatical correctness
- Determines if the input is syntactically well formed
- Guides checking at deeper levels than syntax
- Builds an IR representation of the code

Think of this chapter as the mathematics of diagramming sentences



The Study of Parsing

The process of discovering a *derivation* for some sentence

- Need a mathematical model of syntax — a grammar G
- Need an algorithm for testing membership in $L(G)$
- Need to keep in mind that our goal is building parsers, not studying the mathematics of arbitrary languages

Roadmap for our study of parsing

- 1 Context-free grammars and derivations
- 2 Top-down parsing
 - Generated LL(1) parsers & hand-coded recursive descent parsers
- 3 Bottom-up parsing
 - Generated LR(1) parsers

Today

Lab 2



Specifying Syntax with a Grammar

Context-free syntax is specified with a context-free grammar

$$\begin{array}{l} \text{SheepNoise} \rightarrow \text{SheepNoise } \underline{\text{baa}} \\ \quad \quad \quad | \quad \underline{\text{baa}} \end{array}$$

This *CFG* defines the set of noises sheep normally make

It is written in a variant of Backus-Naur form

Formally, a grammar is a four tuple, $G = (S, N, T, P)$

- S is the *start symbol* (set of strings in $L(G)$)
- N is a set of *nonterminal symbols* (syntactic variables)
- T is a set of *terminal symbols* (words)
- P is a set of *productions or rewrite rules* ($P: N \rightarrow (N \cup T)^+$)

Example due to Dr. Scott K. Warren



Deriving Syntax

We can use the *SheepNoise* grammar to create sentences

- use the productions as *rewriting rules*

Rule	Sentential Form
—	<i>SheepNoise</i>
2	<u>baa</u>

Rule	Sentential Form
—	<i>SheepNoise</i>
1	<i>SheepNoise</i> <u>baa</u>
2	<u>baa</u> <u>baa</u>

Rule	Sentential Form
—	<i>SheepNoise</i>
1	<i>SheepNoise</i> <u>baa</u>
1	<i>SheepNoise</i> <u>baa</u> <u>baa</u>
2	<u>baa</u> <u>baa</u> <u>baa</u>

And so on ...

While this example is cute, it quickly runs out of intellectual steam ...



Why Not Use Regular Languages & DFAs?

Not all languages are regular

($RL's \subset CFL's \subset CSL's$)

You cannot construct DFA's to recognize these languages

- $L = \{p^k q^k\}$ (parenthesis languages)
- $L = \{wcw^r \mid w \in \Sigma^*\}$

Neither of these is a regular language

(nor an RE)

To recognize these features requires an arbitrary amount of context (left or right ...)

But, this issue is somewhat subtle. You can construct DFA's for

- Strings with alternating 0's and 1's
 $(\epsilon \mid 1)(01)^*(\epsilon \mid 0)$
- Strings with an even number of 0's and 1's

RE's can count bounded sets and bounded differences



Limits of Regular Languages

Advantages of Regular Expressions

- Simple & powerful notation for specifying patterns
- Automatic construction of fast recognizers
- Many kinds of syntax can be specified with REs

Example — a regular expression for arithmetic expressions

$Term \rightarrow [a-zA-Z] ([a-zA-Z] | [0-9])^*$

$Op \rightarrow + | - | * | /$

$Expr \rightarrow (Term Op)^* Term$

$([a-zA-Z] ([a-zA-Z] | [0-9])^* (+ | - | * | /))^* [a-zA-Z] ([a-zA-Z] | [0-9])^*$

Of course, this would generate a DFA ...

If REs are so useful ... Why not use them for everything?

⇒ Cannot add parenthesis, brackets, begin-end pairs, ...



Context-free Grammars

What makes a grammar “context free”?

The SheepNoise grammar has a specific form:

$$\begin{array}{l} \textit{SheepNoise} \rightarrow \textit{SheepNoise} \text{ baa } \\ \quad \quad \quad | \text{ baa } \end{array}$$

Productions have a single nonterminal on the left hand side, which makes it impossible to encode left or right context.

⇒ The grammar is context free.

A context-sensitive grammar can have ≥ 1 nonterminal on lhs.

Notice that $L(\textit{SheepNoise})$ is actually a regular language: baa⁺

RL's $\subset$ CFL's



A More Useful Grammar Than Sheep Noise

To explore the uses of CFGs, we need a more complex grammar

0	$Expr$	$\rightarrow$	$Expr Op Expr$
1			<u>number</u>
2			<u>id</u>
3	Op	$\rightarrow$	+
4			-
5			*
6			/

Rule	Sentential Form
—	$Expr$
0	$Expr Op Expr$
2	$\langle id, \underline{x} \rangle Op Expr$
4	$\langle id, \underline{x} \rangle - Expr$
0	$\langle id, \underline{x} \rangle - Expr Op Expr$
1	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle Op Expr$
5	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * Expr$
2	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

- Such a sequence of rewrites is called a *derivation*
- Process of discovering a derivation is called *parsing*

We denote this derivation: $Expr \Rightarrow^* \underline{id} - \underline{num} * \underline{id}$



Derivations

The point of parsing is to construct a derivation

- At each step, we select a NT in the current string to replace
- Different choices can lead to different derivations

Two derivations are of interest

- *Leftmost derivation* — replace leftmost NT at each step
- *Rightmost derivation* — replace rightmost NT at each step

These are the two *systematic* derivations

(We don't care about randomly-ordered derivations!)

The example on the preceding slide was a *leftmost* derivation

- Of course, there is also a *rightmost* derivation
- Interestingly, it turns out to be different



Derivations

The point of parsing is to construct a derivation

A derivation consists of a series of rewrite steps

$$S \Rightarrow \gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow \text{sentence}$$

- Each γ_i is a sentential form
 - If γ contains only terminal symbols, γ is a **sentence** in $L(G)$
 - If γ contains 1 or more non-terminals, γ is a **sentential form**
- To get γ_i from γ_{i-1} , expand some $NT A \in \gamma_{i-1}$ by using $A \rightarrow \beta$
 - Replace the occurrence of $A \in \gamma_{i-1}$ with β to get γ_i
 - In a leftmost derivation, it would be the first $NT A \in \gamma_{i-1}$

A **left-sentential form** occurs in a leftmost derivation

A **right-sentential form** occurs in a rightmost derivation



The Two Derivations for $\underline{x} - \underline{2} * \underline{y}$

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
2	$\langle id, \underline{x} \rangle Op Expr$
4	$\langle id, \underline{x} \rangle - Expr$
0	$\langle id, \underline{x} \rangle - Expr Op Expr$
1	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle Op Expr$
5	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * Expr$
2	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

Leftmost derivation

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
2	<i>Expr Op</i> $\langle id, \underline{y} \rangle$
5	<i>Expr</i> * $\langle id, \underline{y} \rangle$
0	<i>Expr Op Expr</i> * $\langle id, \underline{y} \rangle$
1	<i>Expr Op</i> $\langle num, \underline{2} \rangle$ * $\langle id, \underline{y} \rangle$
4	<i>Expr</i> - $\langle num, \underline{2} \rangle$ * $\langle id, \underline{y} \rangle$
2	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

Rightmost derivation

In both cases, $Expr \Rightarrow^* \underline{id} - \underline{num} * \underline{id}$

- The two derivations produce different parse trees
- The parse trees imply different evaluation orders!

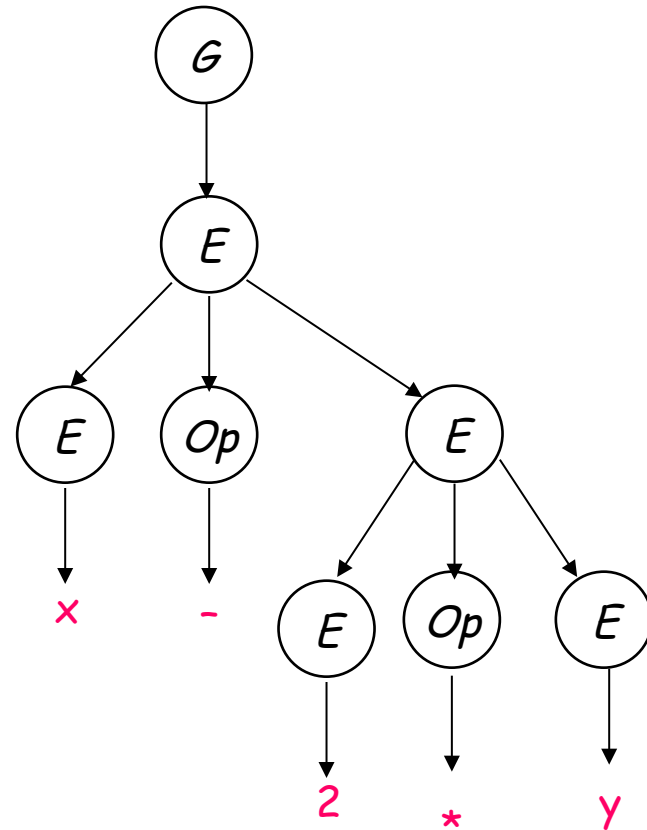


Derivations and Parse Trees

Leftmost derivation

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
2	<i><id,x> Op Expr</i>
4	<i><id,x> - Expr</i>
0	<i><id,x> - Expr Op Expr</i>
1	<i><id,x> - <num,2> Op Expr</i>
5	<i><id,x> - <num,2> * Expr</i>
2	<i><id,x> - <num,2> * <id,y></i>

This evaluates as $x - (2 * y)$



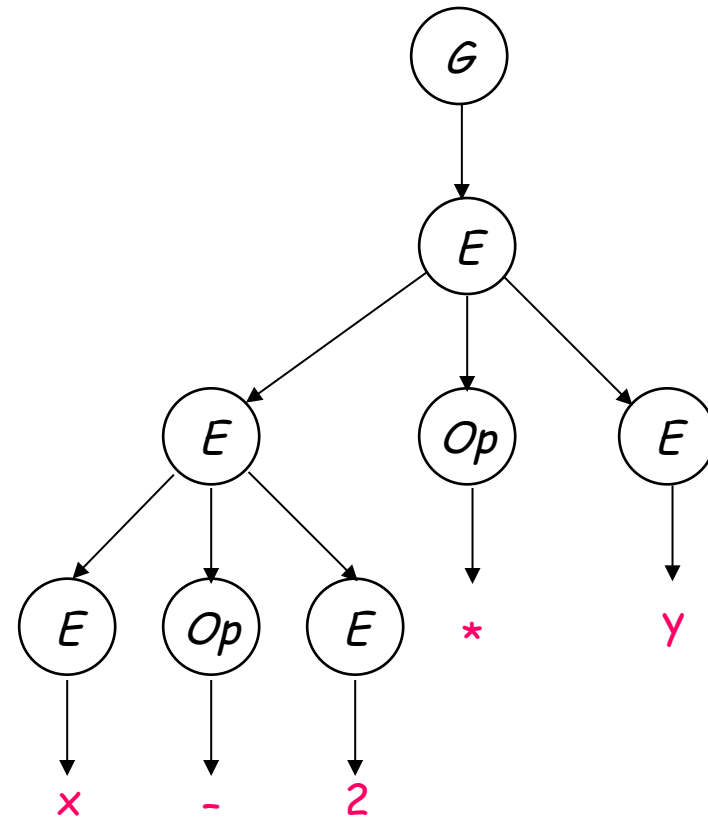


Derivations and Parse Trees

Rightmost derivation

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
2	<i>Expr Op</i> <id, y >
5	<i>Expr</i> * <id, y >
0	<i>Expr Op Expr</i> * <id, y >
1	<i>Expr Op</i> <num, <u>2</u> > * <id, y >
4	<i>Expr</i> - <num, <u>2</u> > * <id, y >
2	<id, <u>x</u> > - <num, <u>2</u> > * <id, y >

This evaluates as $(x - 2) * y$



This ambiguity is NOT good

Derivations and Precedence



*These two derivations point out a problem with the grammar:
It has no notion of precedence, or implied order of evaluation*

To add precedence

- Create a nonterminal for each *level of precedence*
- Isolate the corresponding part of the grammar
- Force the parser to recognize high precedence subexpressions first

For algebraic expressions

- Parentheses first (level 1)
- Multiplication and division, next (level 2)
- Subtraction and addition, last (level 3)



Derivations and Precedence

Adding the standard algebraic precedence produces:

	0	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
level 3	1	<i>Expr</i>	$\rightarrow$	<i>Expr</i> + <i>Term</i>
	2			<i>Expr</i> - <i>Term</i>
	3			<i>Term</i>
level 2	4	<i>Term</i>	$\rightarrow$	<i>Term</i> * <i>Factor</i>
	5			<i>Term</i> / <i>Factor</i>
	6			<i>Factor</i>
level 1	7	<i>Factor</i>	$\rightarrow$	(<i>Expr</i>)
	8			<u>number</u>
	9			<u>id</u>

This grammar is slightly larger

- Takes more rewriting to reach some of the terminal symbols
- Encodes expected precedence
- Produces same parse tree under leftmost & rightmost derivations
- Correctness trumps the speed of the parser

*Let's see how it parses $x - 2 * y$*

Cannot handle precedence in an RE for expressions

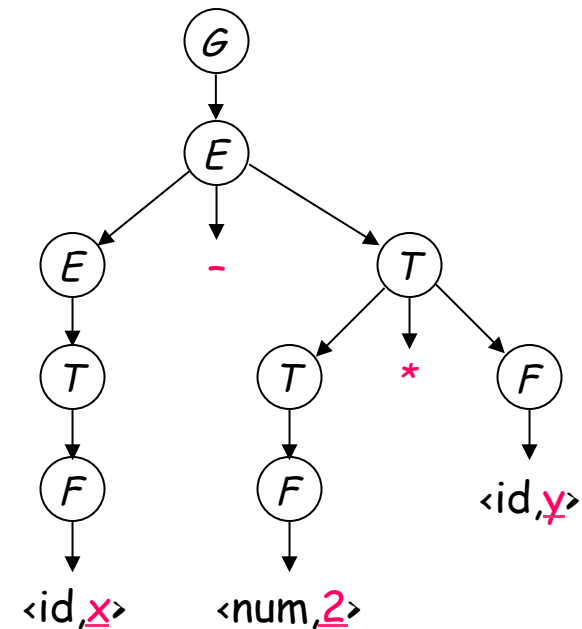
Introduced parentheses, too (beyond power of an RE)



Derivations and Precedence

Rule	Sentential Form
—	Goal
0	Expr
2	Expr - Term
4	Expr - Term * Factor
9	Expr - Term * <id,y>
6	Expr - Factor * <id,y>
8	Expr - <num,2> * <id,y>
3	Term - <num,2> * <id,y>
6	Factor - <num,2> * <id,y>
9	<id,x> - <num,2> * <id,y>

The rightmost derivation



Its parse tree

It derives $x - (2 * y)$, along with an appropriate parse tree.

Both the leftmost and rightmost derivations give the same expression, because the grammar directly and explicitly encodes the desired precedence.



Ambiguous Grammars

Let's leap back to our original expression grammar.
It had other problems.

0	$Expr \rightarrow Expr\ Op\ Expr$
1	<u>number</u>
2	<u>id</u>
3	$Op \rightarrow +$
4	-
5	*
6	/

Rule	Sentential Form
—	$Expr$
0	$Expr\ Op\ Expr$
②	$\langle id, \underline{x} \rangle\ Op\ Expr$
4	$\langle id, \underline{x} \rangle - Expr$
0	$\langle id, \underline{x} \rangle - Expr\ Op\ Expr$
1	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle\ Op\ Expr$
5	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * Expr$
2	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

- This grammar allows multiple leftmost derivations for $\underline{x} - \underline{2} * \underline{y}$
- Hard to automate derivation if > 1 choice
- The grammar is *ambiguous*

Different choice
than the first time



Two Leftmost Derivations for $x - 2 * y$

The Difference:

- Different productions chosen on the second step

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
②	<i><id,x> Op Expr</i>
4	<i><id,x> - Expr</i>
0	<i><id,x> - Expr Op Expr</i>
1	<i><id,x> - <num,2> Op Expr</i>
5	<i><id,x> - <num,2> * Expr</i>
1	<i><id,x> - <num,2> * <id,y></i>

Original choice

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
①	<i>Expr Op Expr Op Expr</i>
2	<i><id,x> Op Expr Op Expr</i>
4	<i><id,x> - Expr Op Expr</i>
1	<i><id,x> - <num,2> Op Expr</i>
5	<i><id,x> - <num,2> * Expr</i>
2	<i><id,x> - <num,2> * <id,y></i>

New choice

- Both derivations succeed in producing $x - 2 * y$



Two Leftmost Derivations for $x - 2 * y$

The Difference:

- Different productions chosen on the second step

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
2	<i><id,x> Op Expr</i>
4	<i><id,x> - Expr</i>
0	<i><id,x> - Expr Op Expr</i>
1	<i><id,x> - <num,2> Op Expr</i>
5	<i><id,x> - <num,2> * Expr</i>
2	<i><id,x> - <num,2> * <id,y></i>

Original choice

Rule	Sentential Form
—	<i>Expr</i>
0	<i>Expr Op Expr</i>
0	<i>Expr Op Expr Op Expr</i>
2	<i><id,x> Op Expr Op Expr</i>
4	<i><id,x> - Expr Op Expr</i>
1	<i><id,x> - <num,2> Op Expr</i>
5	<i><id,x> - <num,2> * Expr</i>
2	<i><id,x> - <num,2> * <id,y></i>

New choice

Different choices in same situation, again

Remember nondeterminism?



Ambiguous Grammars

Definitions

- If a grammar has more than one leftmost derivation for a single *sentential form*, the grammar is *ambiguous*
- If a grammar has more than one rightmost derivation for a single sentential form, the grammar is *ambiguous*
- The leftmost and rightmost derivations for a sentential form may differ, even in an unambiguous grammar
 - However, they must have the same parse tree!

Classic example — the *if-then-else* problem

$$\begin{aligned} \text{Stmt} \rightarrow & \text{if Expr then Stmt} \\ & | \text{if Expr then Stmt else Stmt} \\ & | \dots \text{other stmts} \dots \end{aligned}$$

This ambiguity is inherent in the grammar

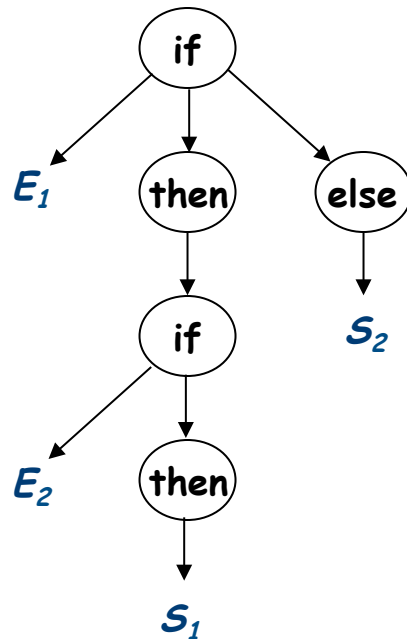


Ambiguity

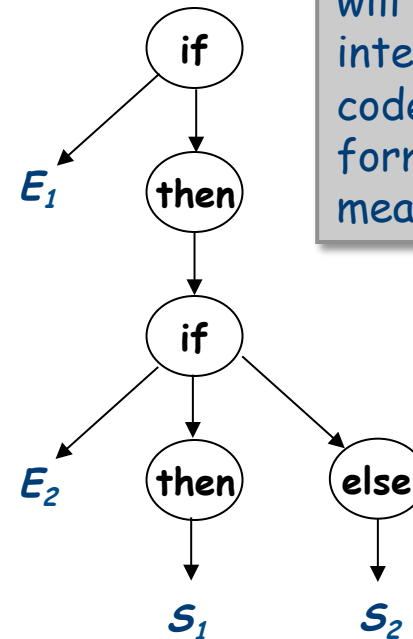
This sentential form has two derivations

if $Expr_1$ then if $Expr_2$ then $Stmt_1$ else $Stmt_2$

Part of the problem is that the structure built by the parser will determine the interpretation of the code, and these two forms have different meanings!



*production 2, then
production 1*



*production 1, then
production 2*

Ambiguity

The grammar forces the structure to match the desired meaning.



Removing the ambiguity

- Must rewrite the grammar to avoid generating the problem
- Match each else to innermost unmatched if (*common sense rule*)

0	<i>Stmt</i>	→	<u>if</u> <i>Expr</i> <u>then</u> <i>Stmt</i>
1			<u>if</u> <i>Expr</i> <u>then</u> <i>WithElse</i> <u>else</u> <i>Stmt</i>
2			<i>Other Statements</i>
3	<i>WithElse</i>	→	<u>if</u> <i>Expr</i> <u>then</u> <i>WithElse</i> <u>else</u> <i>WithElse</i>
4			<i>Other Statements</i>

With this grammar, example has only one rightmost derivation

Intuition: once into *WithElse*, we cannot generate an unmatched else
... a final if without an else can only come through rule 2 ...



Ambiguity

if $Expr_1$ then if $Expr_2$ then $Stmt_1$ else $Stmt_2$

Rule	Sentential Form
—	$Stmt$
0	<u>if</u> $Expr$ <u>then</u> $Stmt$
1	<u>if</u> $Expr$ <u>then</u> <u>if</u> $Expr$ <u>then</u> $WithElse$ <u>else</u> $Stmt$
2	<u>if</u> $Expr$ <u>then</u> <u>if</u> $Expr$ <u>then</u> $WithElse$ <u>else</u> S_2
4	<u>if</u> $Expr$ <u>then</u> <u>if</u> $Expr$ <u>then</u> S_1 <u>else</u> S_2
?	<u>if</u> $Expr$ <u>then</u> <u>if</u> E_2 <u>then</u> S_1 <u>else</u> S_2
?	<u>if</u> E_1 <u>then</u> <u>if</u> E_2 <u>then</u> S_1 <u>else</u> S_2

Other productions to derive $Exprs$

This grammar has only one rightmost derivation for the example



Deeper Ambiguity

Ambiguity usually refers to confusion in the CFG

Overloading can create deeper ambiguity

$a = f(17)$

In many Algol-like languages, f could be either a function or a subscripted variable

Disambiguating this one requires context

- Need values of declarations
- Really an issue of *type*, not context-free syntax
- Requires an extra-grammatical solution (not in CFG)
- Must handle these with a different mechanism
 - Step outside grammar rather than use a more complex grammar



Ambiguity - the Final Word

Ambiguity arises from two distinct sources

- Confusion in the context-free syntax (*if-then-else*)
- Confusion that requires context to resolve (*overloading*)

Resolving ambiguity

- To remove context-free ambiguity, rewrite the grammar
- To handle context-sensitive ambiguity takes cooperation
 - Knowledge of declarations, types, ...
 - Accept a superset of $L(G)$ & check it by other means[†]
 - This is a language design problem

Sometimes, the compiler writer accepts an ambiguous grammar

- Parsing techniques that “do the right thing”
- *i.e.*, always select the same derivation